

```

1  FT_IRC - CODE WALKTHROUGH
2  =====
3
4  BIG PICTURE ARCHITECTURE
5  -----
6  This is a single-threaded, non-blocking IRC server built with poll().
7
8  main.cpp          -> creates a Server, parses argv, runs it
9  server.hpp/.cpp   -> socket setup, poll() loop, accept/read/write, register commands
10 server_cmds.cpp    -> PASS/NICK/USER (registration) + command dispatcher
11 server_cmds_channel.cpp -> JOIN/PART/PRIVMSG + channel helpers
12 server_channel2.cpp -> KICK/INVITE/TOPIC
13 client.hpp/.cpp    -> per-connection state (buffers, nick, user, registered)
14 channel.hpp/.cpp   -> per-channel state (members, operators, invited, topic)
15 parsing.hpp/.cpp   -> turns a raw IRC line into a Message{command, params}
16
17 Everything revolves around ONE poll() loop watching all client sockets at
18 once (no threads).
19
20 -----
21 parsing.hpp / parsing.cpp
22 -----
23 Turns a raw line like "PRIVMSG #chan :hello world" into:
24
25     struct Message { std::string command; std::vector<std::string> params; };
26
27 Logic: skip leading spaces -> read command until space -> then loop reading
28 params, splitting on spaces, EXCEPT once it hits a ':', everything after
29 that becomes one single last param (the "trailing" IRC convention). This is
30 exactly how real IRC parses lines, minus prefixes (":nick!user@host COMMAND
31 ...") since this server never receives lines with a leading prefix from
32 clients.
33
34 -----
35 client.hpp / client.cpp
36 -----
37 One Client object per connected socket. Holds:
38 - fd_client      : the socket fd
39 - buffer         : INPUT accumulator (raw bytes read from recv() that
40                   haven't formed a full line yet)
41 - outp_Buffer    : OUTPUT queue (bytes waiting to be send()'d)
42 - pass/nick/user/registered : registration state
43
44 Key methods:
45 - append_to_buffer      : appends raw recv'd bytes
46 - is_line_complete/extract_line : since TCP is a stream, a single recv()
47   might contain 0, 1, or several IRC lines (or a partial one). This buffers
48   until a '\n' shows up, then extracts up to it, strips a trailing '\n'

```

```

49     (IRC lines end \r\n), and removes it from the buffer.
50 - queue_output/consume_outp  : same idea for outgoing data (since send()
51     can also write partial data on a non-blocking socket).
52
53 -----
54 channel.hpp / channel.cpp
55 -----
56 One Channel object per #channel. Holds three vector<int> of fds: members,
57 operators, invited, plus a topic string.
58
59 Straightforward: add/remove/check membership, operator status, invite
60 status. retire_client(fd) ("remove client" in French) is the one-stop
61 cleanup called when a client leaves/disconnects/gets kicked – it strips
62 them from all three lists.
63
64 NOTE: no channel modes exist yet (no +i, +t, +k, +l, +o storage). That's
65 exactly the gap to fill for MODE.
66
67 -----
68 server.hpp
69 -----
70 The class declaration – the map of what exists:
71 - Socket/lifecycle : config_socket, start_serv, run_serv, accept_new_client,
72     disconnect_client
73 - I/O               : lire_depuis_client (read), send_repons_to_client
74     (write), queue_response
75 - Registration      : handl_pass/nick/user, try_register, is_nick_taken
76 - Channel commands  : handl_join/part/privmsg/kick/invite/topic
77 - Helpers           : find_channel, find_client_fd_by_nickname,
78     diffusion_msg_to_channel (broadcast), supp_from_all_channels,
79     client_prefix (builds nick!user@host)
80 - Numeric replies   : send_numeric_reply, format_code
81
82 clients is a map<int fd, Client>, channels is a map<string name, Channel>.
83 poll_fds is the vector passed straight to poll().
84
85 -----
86 server.cpp
87 -----
88 The engine room:
89
90 1. pars_port / check_key_serv – validate argv (port 1-65535 numeric,
91     password non-empty/no whitespace).
92 2. config_socket – socket() -> SO_REUSEADDR -> non-blocking -> bind() ->
93     listen().
94 3. start_serv – calls config_socket, pushes the listening fd into
95     poll_fds watching POLLIN.
96 4. accept_new_client – accept(), set non-blocking, push a new pollfd,

```

```

97     create a Client in the map.
98 5. disconnect_client – removes from channels, clients map, closes fd,
99    erases from poll_fds.
100 6. lire_depuis_client – recv() into a stack buffer, append to the
101    Client's buffer, then loop extracting/parsing/dispatching every
102    complete line.
103 7. send_repons_to_client – flushes queued output; clears POLLOUT
104    interest once empty.
105 8. run_serv – the actual event loop:
106     poll() blocks until something is ready
107     for each watched fd:
108         listening socket + POLLIN -> accept
109         client + (POLLHUP|POLLERR) -> disconnect
110         client + POLLIN -> read
111         client + POLLOUT -> write
112
113     Note the "continue" after disconnect_client/failed read/write – that's
114     because erasing from poll_fds shifts indices, so it must NOT increment
115     i in that iteration. This is a common bug spot if you add code here
116     later – be careful not to use a stale poll_fds[i] after any function
117     that might disconnect the client.

```

```

118 -----
119 server_cmds.cpp
120 -----
121
122 - queue_response – writes to a client's outp_Buffer and flips on POLLOUT
123   for their fd so run_serv knows to try sending.
124 - format_code/send_numeric_reply – builds standard ":ircserv 4NN target
125   :text\r\n" replies.
126 - process_command – the dispatcher (if/else if chain on msg.command).
127   THIS IS WHERE YOU'LL ADD MODE.
128 - try_register – registration only completes once PASS succeeded, NICK
129   set, and USER set; then sends the welcome (numeric 001-ish, though it's
130   hardcoded as 1).
131 - handl_pass/nick/user – straightforward, with is_nick_taken guarding
132   uniqueness.

```

```

133 -----
134 server_cmds_channel.cpp
135 -----
136
137 - client_prefix – builds the nick!user@host string used in relayed
138   messages.
139 - find_channel / find_client_fd_by_nickname – lookups.
140 - diffusion_msg_to_channel – broadcasts a raw line to every member fd
141   except one excluded fd (-1 = exclude nobody).
142 - handl_join – validates registration + '#' prefix, creates channel if
143   needed, first joiner becomes operator, broadcasts JOIN, replies with
144   353/366 (NAMES list).

```

```

145 - handl_part - broadcasts PART, removes client, deletes channel if empty.
146 - handl_privmsg - routes to a channel (#...) or a nick.
147 - supp_from_all_channels - called on disconnect; removes the fd from
148     every channel it's in, deleting empty channels.
149
150 -----
151 server_channel2.cpp
152 -----
153 - handl_kick - checks registration, params, channel exists, sender is
154     member+operator, target is a member -> broadcasts KICK, removes target.
155 - handl_invit - checks operator status, target exists and isn't already
156     a member -> adds to invited list, notifies target, replies 341.
157 - handl_topic - no args = query current topic (331/332); with arg = set
158     + broadcast.
159
160 =====
161 WHERE MODE FITS (next step)
162 =====
163 MODE is missing entirely - no handler, no storage in Channel for modes.
164 The subject usually requires at least: i (invite-only), t (topic
165 restricted to ops), k (key/password), o (operator grant/revoke), l (user
166 limit).
167
168 Plan (not full code - implement it yourself, matching existing style):
169
170 1. channel.hpp/channel.cpp - add state:
171
172     bool invite_only;        // +i
173     bool topic_restricted;   // +t
174     std::string key;         // +k (empty = no key)
175     bool has_key;
176     int user_limit;          // +l
177     bool has_limit;
178
179     Plus getters/setters, matching the style already used (is_invited,
180     set_topic, etc.).
181
182 2. server.hpp - declare:
183
184     void handl_mode(int fd, const Message &msg);
185
186     and register it in process_command's if/else chain (server_cmds.cpp),
187     same pattern as the others.
188
189 3. New handler logic (put it in a new file, e.g. server_mode.cpp, and
190     add it to the Makefile's SRC):
191
192     - Parse msg.params[0] = channel, msg.params[1] = mode string like

```

```
193     "+o" or "-i" or "+k", remaining params = arguments consumed
194     left-to-right for modes that need one (k, o, l; +l needs an arg,
195     -l doesn't; same idea for k).
196 - Check registration, channel exists, sender is a member and (for
197   restricted modes) an operator – reuse the 442/482 numeric pattern
198   already used in handl_kick.
199 - Walk the mode string character by character, tracking a +/- sign
200   flag as you go (that's literally how the IRC spec's MODE string
201   works: "+ol" reads as +o then +l, "+o-l" flips mid-string).
202 - Apply changes to the Channel object via new setters.
203 - Reuse handl_join's logic: once +i exists, JOIN must check
204   channel->is_invite_only() && !channel->is_invited(fd) -> reject
205   with 473. Once +k exists, JOIN needs a key param to compare. Once
206   +l exists, check members.size() >= limit. So implementing MODE
207   also means going back into handl_join to ENFORCE the modes it sets.
208 - Broadcast the change with a MODE line, same broadcast pattern as
209   handl_topic/handl_kick (diffusion_msg_to_channel).
210 - Reply-on-query: "MODE #chan" with no mode string -> typically
211   replies 324 with the current mode string.
```

212

```
213 That's a self-contained, well-scoped addition that fits the existing
214 style perfectly – build it exactly the way handl_kick/handl_invit/
215 handl_topic are structured (same error-checking order, same
216 numeric-reply pattern) so it reads consistently with the rest of the
217 codebase.
```

218

```
219 =====
```

```
220 ABOUT "FILE TRANSFER AND BOT"
```

```
221 =====
```

```
222 Worth flagging: DCC (file transfer, and most IRC bots) is NOT something
223 the server implements. DCC file transfer is a CLIENT-TO-CLIENT direct TCP
224 connection – the IRC server only relays a PRIVMSG containing a "DCC SEND"
225 control string (with an IP/port encoded in it); the actual file bytes
226 never touch the server. A "bot" is just an ordinary IRC client (something
227 that connects to your server with a NICK/USER and reacts to messages) –
228 not a server feature either.
```

229

```
230 So if your subject/grading actually requires those, they'd be SEPARATE
231 programs: a small client that connects to ircserv like nc does now, sends
232 normal NICK/USER/PRIVMSG commands, and (for DCC) opens its own listening
233 socket to transfer a file. That work doesn't touch any of the 14 server
234 files above – it would be its own small project.
```

235